Agentic Project Management

BSc Thesis Case Study

Participation Guide

Konstantinos Chaidemenos

National and Kapodistrian University of Athens

Department of Informatics and Telecommunications

1. Welcome

Thank you for taking part in this research. The study compares two AI-assisted software development frameworks, Agentic Project Management (APM) and GitHub Spec-kit, on a fixed coding task. Your contribution helps build the empirical evidence base for the thesis.

You are not being evaluated. The frameworks are. Whatever you produce during your sessions, including incomplete or non-functional code, is valuable data.

1.1. Data privacy and anonymity

Submissions are handled with strict confidentiality. No personal identifier is linked to your data. You have been assigned a participant ID (e.g., `P001`); use it in your submission filenames as described later. AI interaction logs are encrypted with a key held only by the research team before being added to the published research dataset. The dataset's structure is transparent; the contents of your sessions remain private.

By proceeding, you consent to the anonymous collection and use of this data as described.

1.2. Before you start

You have been provided separately with:

- your participant ID;
- your per-session framework assignment (which framework to use in session 1 and which in session 2);
- the Claude Code credentials you will use during the sessions, alongside short setup material describing how to apply them inside your Linux environment.

Keep these to hand. They do not appear anywhere in this guide or in the participant package.

Read the official documentation of your assigned framework before each session begins; a short throwaway run on an unrelated repository is recommended for first-time familiarity.

- Agentic Project Management (APM): https://agentic-project-management.dev.
- GitHub Spec-kit: https://github.github.io/spec-kit/.

2. Study overview

2.1. What you do

You will complete the same coding task twice, in two separate three-hour sessions, each using a different framework. The two sessions can run back to back, on the same day, or on different days. They must not overlap, and you must approach the second session as if the first had not happened, working from a fresh checkout of the codebase.

2.2. Time

Each session is strictly time-boxed to three hours. Stop at the three-hour mark whether or not the task is complete. Submitting unfinished work is an expected and valid outcome.

2.3. What you submit

After each session you produce one zip file containing two artefacts at its root:

- `solution.patch`: a unified diff of your work against a pinned starting commit of the implementation codebase.
- `transcripts/`: the AI interaction logs. One `.jsonl` per main session at the top level, plus a subdirectory of the same uuid for each carrying the `subagents/` and `tool-results/` records Claude Code wrote alongside that session.

The zip filename follows the strict pattern `{PID}_S{1|2}_{apm|speckit}.zip`, e.g., `P001_S1_apm.zip`.

You also answer a short post-session questionnaire about your experience with the framework. The questionnaire and the zip go through a single submission form, whose URL is given later in Section 8.3, step 5.

3. Participant package

This guide ships inside a participant-package zip alongside the task specification and a helper skill that drives the participant's setup and per-session lifecycle. Its layout:

```
participant-package/
|-- participant-guide-cc.pdf       # this document
|-- task/
|   |-- PRD.md                     # Product Requirements Document
|   `-- PROMPT.md                  # default opening prompt for the AI
`-- .claude/
    `-- skills/
        `-- bsc-apm-study-helper/  # Claude Code skill (see Section 3.1)
            |-- SKILL.md
            `-- references/...
```

`PRD.md` is the sole source of truth for what to implement.

3.1. AI-assisted setup

The package also ships a Claude Code skill at `.claude/skills/bsc-apm-study-helper/` as an alternative to the hands-on instructions in the rest of this guide. The skill drives setup,

the per-session lifecycle, the between-sessions reset, and the optional cleanup, asking for your confirmation before running anything. Two ways to use it:

From your host machine, with your AI assistant. Whatever AI assistant you have on your host, it can access the skill as plain markdown after the package is unzipped, and walk you through host-side setup, including launching a Linux VM. The study coordinator's invitation includes a starter prompt you can paste into your AI assistant to get started; it fetches and unzips the package, reads the skill, and continues from there.

From inside the Linux environment, with Claude Code installed. Fetch and unzip the package inside the Linux environment, open Claude Code with the participant-package directory as its working directory, and the skill loads natively.

The skill then drives the rest of the work in conversation with you, guiding you through this setup.

Both paths land on the same workspace and produce the same submission. The hands-on instructions in the rest of this guide cover every operation the skill walks through; choose whichever you prefer.

Note. The skill is project-scoped to the participant-package directory, not user-level. It is therefore not loaded in your task's workspace, only in a Claude Code session opened from the participant-package directory. Do not copy it to `~/.claude/skills/` or to your workspace's `.claude/skills/`; doing so would defeat that isolation.

Warning. The skill is for logistics only. It is instructed to refuse questions about the implementation task or the frameworks themselves and to redirect you to their documentation, but the agent may still slip; if it does, the responsibility to disregard such answers lies with you. Acting on, paraphrasing, or carrying any task or framework content the helper produced into a session contaminates your study data.

4. System requirements

4.1. Operating system

The study runs inside a dedicated Linux virtual machine, freshly created for the study. The tool depends on your host operating system: Lima on macOS and on native Linux; WSL2 with a new Ubuntu distribution on Windows.

Note. Create a new VM for the study; do not reuse a pre-existing VM or WSL distribution you use for other work. Prior Claude Code state, project history, or partial setups would interfere with transcript collection and authentication, and the post-session evaluation depends on a clean baseline.

macOS and native Linux: Use Lima (https://lima-vm.io). A free, open-source, CLI-driven VM manager that brings up an Ubuntu LTS VM in one command. Install Lima following the official instructions at https://lima-vm.io/docs/installation/, which cover Homebrew on macOS and distribution packages or release tarballs on Linux. Then:

```
limactl start --name=task --cpus=4 --memory=8 --disk=20 \
    --tty=false template:ubuntu-24.04
limactl shell task
```

`limactl shell task` opens a shell inside the VM but lands you at the host directory you launched it from, not the VM's own home; `cd ~` before any download or extraction so files land on the VM filesystem.

Windows: Use WSL2 (https://learn.microsoft.com/en-us/windows/wsl/). Install WSL2 following the official Microsoft instructions at https://learn.microsoft.com/en-us/windows/wsl/install. Then, from an elevated PowerShell on the Windows host, create the `task` distribution from the official Ubuntu rootfs tarball and launch it:

```
$rootfs = "ubuntu-noble.tar.gz"
$url = "https://cloud-images.ubuntu.com/wsl/noble/current/" +
       "ubuntu-noble-wsl-amd64-wsl.rootfs.tar.gz"
Invoke-WebRequest -Uri $url -OutFile $rootfs
mkdir C:\WSL -Force | Out-Null
wsl --import task C:\WSL\task $rootfs
wsl -d task
```

The first shell lands you in as `root` (the imported rootfs has no non-root user yet); operating as `root` is fine for the rest of the study, and sudo-prefixed commands in later sections work as expected under root.

> **Warning.** Docker Desktop is not a substitute. The mock environment described in Section 7 runs Docker Engine *inside* the VM, not on the host.

4.2. System resources

The Linux environment needs at least 4 vCPUs, 8 GB of RAM, and a 20 GB virtual disk; the launch commands above provision these for VM users. After Docker images are loaded (about 1.6 GB for the openeclass container), at least 5 GB of free disk should remain for the workspace, Docker state, and transient files.

4.3. Tools and libraries

The case-study work needs a container runtime (Docker Engine with Compose v2), a C build chain (`gcc`, `make`, `valgrind`, `pkg-config`, plus the `libcurl` and `libxml2` development headers),

two scripting runtimes (Python 3.10 or newer with `uv`, and Node.js with `npm`), and a handful of command-line utilities (`git`, `curl`, `tar`, `zip`, `unzip`, `openssl`, `bash` 3.2 or newer, and `gh`). Install Docker first via the official convenience script. Download it before running so you can read it:

```
curl -fsSL https://get.docker.com -o /tmp/install-docker.sh
sudo sh /tmp/install-docker.sh
sudo usermod -aG docker $USER
```

The new `docker` group membership takes effect on your next login or after `newgrp docker`. Install everything else with `apt` and the official `uv` installer:

```
sudo apt-get update
sudo apt-get install -y \
    build-essential pkg-config valgrind \
    libcurl4-openssl-dev libxml2-dev \
    git curl tar zip unzip openssl gh python3 python3-pip npm
curl -LsSf https://astral.sh/uv/install.sh -o /tmp/install-uv.sh
sh /tmp/install-uv.sh
```

If `uv -version` reports `command not found` immediately after the install, run `source ~/.local/bin/env` once or open a new login shell so the installer's PATH update takes effect. The C minimums are `gcc` 9.0 (for C11), `libcurl` 7.81.0, and `libxml2` 2.9.13; Ubuntu 22.04 and 24.04 both satisfy them. Verify the toolchain:

```
gcc --version
pkg-config --modversion libcurl
pkg-config --modversion libxml-2.0
valgrind --version
python3 --version
node --version
npm --version
uv --version
```

5. AI tools

The session uses a coding-agent CLI (Claude Code) and one of two AI-assisted development frameworks (APM or Spec-kit). The participant package does not bundle any of them. Install Claude Code once before either session; the two framework subsections below cover their own install models (APM ships as a global npm package; Spec-kit is invoked via `uvx` from a release tag on every call, with no persistent install needed). Set up whichever framework is assigned to your current session before that session begins. The system requirements in Section 4 cover the underlying tools all three depend on.

5.1. Claude Code

The recommended install path is Anthropic's shell installer:

```
curl -fsSL https://claude.ai/install.sh -o /tmp/install-cc.sh
bash /tmp/install-cc.sh
claude --version
```

Claude Code `2.1.126` is the floor verified for the study; anything newer is fine.

The official instructions at https://docs.claude.com/en/docs/claude-code cover other install paths. Authentication itself runs once, inside this Linux environment; follow the setup material the study coordinator gave you separately (see Section 1.2) to apply it. The credentials and the setup material live inside this VM from that point on; do not reuse them elsewhere.

5.2. Agentic Project Management (APM)

APM ships as an npm package; the CLI is invoked as `apm`.

```
sudo npm install -g agentic-pm
apm --version
```

5.3. GitHub Spec-kit

Spec-kit runs through `uvx` from a release tag of the upstream repository; the CLI is invoked as `specify`, prefixed by the same `uvx --from` expression on every invocation. The study does not pin a specific tag: open https://github.com/github/spec-kit/releases, pick the current release, and use that tag for every Spec-kit command in your session. `v0.8.5` is the floor verified for the study; anything newer is fine.

```
uvx --from git+https://github.com/github/spec-kit.git@<tag> \
    specify --help
```

6. Workspace

The participant package's `task/` directory holds `PRD.md` (the requirements you implement against) and `PROMPT.md` (the message you give the AI to open the session). The *workspace* is the directory inside your Linux VM in which you open Claude Code; it is the AI's working directory for the session, and it must end up with this exact layout:

```
<workspace>/
|-- PRD.md
|-- PROMPT.md
```

```
|-- eclass-mcp-server/      # implementation codebase
`-- openeclass/             # legacy PHP codebase (read-only reference)
```

Create the workspace as a fresh directory of your choice on the Linux filesystem (`~/workspace/` is a reasonable default), copy `PRD.md` and `PROMPT.md` into it from the participant package's `task/` directory, and clone the two repositories alongside them following the next subsection.

6.1. Cloning the codebases

Clone `eclass-mcp-server` and check out the pinned commit `dbd2d16`. This commit is the patch baseline: every diff you submit is computed against it.

```
cd <workspace>
git clone https://github.com/sdi2200262/eclass-mcp-server.git
cd eclass-mcp-server
git checkout dbd2d16
uv sync --dev --all-extras
cd ..
```

Clone `openeclass` at its pinned commit `e8b3329`. The AI treats this codebase as a read-only reference; you should not modify it.

```
cd <workspace>
git clone https://github.com/gunet/openeclass.git
cd openeclass
git checkout e8b3329
cd ..
```

7. Mock environment

The task integrates with openeclass through a single-sign-on flow, both for the existing Python MCP (Model Context Protocol) server it expands and for the C replica it asks for. The study ships a self-contained *mock environment* that you run locally during each session: a real openeclass instance paired with a mock SSO service whose flow matches `eclass-mcp-server`'s integration, pre-loaded with a fixed test data set so every participant's session runs against identical state. The openeclass instance inside this mock is the same release the read-only `openeclass/` checkout in your workspace points at, packaged as a docker image; reading the `openeclass/` tree is the most direct way to understand how the live mock behaves.

7.1. Download and install

The mock environment ships from the `sdi2200262/bsc-apm-case-study-task` GitHub repository as two architecture-specific releases, each with a single tarball asset:

- Tag `bsc-apm-case-study-task` ships `bsc-apm-amd64.tar.gz` for x86_64 hosts (most Intel and AMD Linux machines, and the Linux VM on an Intel Mac).
- Tag `bsc-apm-case-study-task-arm64` ships `bsc-apm-arm64.tar.gz` for arm64 hosts (Apple Silicon Macs running a Linux VM, arm64 Linux servers, Raspberry Pi 4/5, etc.).

Identify your architecture with `uname -m`: `x86_64` means amd64, `aarch64` or `arm64` means arm64. Pick a prefix on disk for the mock environment, separate from your workspace (the rest of this guide writes `<mock>/` for that path; `~/.bsc-apm` is a reasonable default). Run *one* of the two download commands below, the one that matches your architecture:

```
mkdir -p <mock> && cd <mock>

# x86_64 hosts:
curl -L -o bsc-apm-amd64.tar.gz \
    https://github.com/sdi2200262/bsc-apm-case-study-task/releases/\
download/bsc-apm-case-study-task/bsc-apm-amd64.tar.gz

# arm64 hosts:
curl -L -o bsc-apm-arm64.tar.gz \
    https://github.com/sdi2200262/bsc-apm-case-study-task/releases/\
download/bsc-apm-case-study-task-arm64/bsc-apm-arm64.tar.gz
```

Then unpack and bring the stack up. `./install` loads the bundled Docker images, generates self-signed TLS certificates into `./certs/`, and writes a pre-configured `.env` alongside. `./scripts/up` brings the three containers up, performs openeclass's first-time install on first run, and applies the test data; this takes a few minutes on a clean host.

```
tar -xzf bsc-apm-*.tar.gz
./install
./scripts/up
```

The stack listens on two URLs once it is ready:

- `http://localhost/` for the openeclass instance.
- `https://localhost:18443/` for the mock authentication service.

Ports 80 and 18443 must be free on the Linux environment. Port 80 is the default HTTP port and is more commonly held by another service than 18443 is; if either is bound, stop the holder before running `./scripts/up`. The helper skill walks through identifying and stopping the holder if needed.

7.2. Lifecycle commands

Once installed, manage the environment from its prefix:

- `./scripts/up`: bring the stack up. Idempotent.
- `./scripts/down`: stop the stack; named volumes survive.
- `./scripts/status`: probe each service from the mock side and report whether the stack itself is healthy.
- `./scripts/verify-mcp --mcp-root <path>`: consumer-side complement to `status`; verifies that an `eclass-mcp-server` checkout authenticates against the mock with the `.env` and `certs/` it has been given. Runs only against the pinned baseline (Section 6.1), at initial setup or after a between-sessions reset.
- `./scripts/logs`: tail container logs.
- `./scripts/reset`: drop database state and re-apply the test data.

You should not need to reset during a session. If you do (for instance, after a destructive experiment from the AI), `./scripts/reset` returns the environment to its initial state.

7.3. MCP server configuration

The `.env` and `certs/` that `./install` wrote are everything the implementation needs to authenticate against the mock. Copy both into the `eclass-mcp-server/` checkout, preserving the `certs/` subdirectory:

```
cp <mock>/.env <workspace>/eclass-mcp-server/.env
cp -r <mock>/certs <workspace>/eclass-mcp-server/certs
```

The `.env` references the certificate by relative path, so no further configuration is required. After the copy, run `<mock>/scripts/verify-mcp --mcp-root <workspace>/eclass-mcp-server` (Section 7.2) to confirm the wiring; it prints `wiring ok` on success.

7.4. Uninstall

To remove the mock environment from your machine after the study, stop the stack and drop its volumes (`down -v`), remove the loaded images, and delete the prefix:

```
docker compose -f <mock>/compose.yaml down -v
docker rmi bsc-apm/openeclass:dev bsc-apm/mock-cas:dev
rm -rf <mock>
```

8. Session workflow

A session has three phases: starting, working, and wrapping up. Between your two sessions, run the reset at the end of this section to return the workspace and the transcript store to the same state the setup steps above produced for session 1.

8.1. Starting a session

The session begins the moment you send your first message in the workspace Claude Code session; that timestamp is your wall-clock start time, not the moment you launch `claude`. Inside the workspace, run your assigned framework's workspace bootstrap (`apm init` for APM; `uvx -from git+https://github.com/github/spec-kit.git@<tag> specify init -here` for Spec-kit, using the same `<tag>` picked at install time), launch Claude Code, and send the framework's first message per its official documentation (Section 1.2).

8.2. Working on the task

Work for up to three hours and stop at the three-hour mark whether or not you have finished. Use only the framework assigned to your current session and follow its documented workflow as written; the study compares frameworks as documented, not as adapted on the fly. The mock environment running locally is the same surface the post-session evaluation runs against, so testing your implementation against it as you go is the most direct way to check it against the PRD. Do not delete any `.jsonl` file from your workspace's transcript directory; those are your transcripts and are needed in your submission.

8.3. Wrapping up

1. Create the patch. From inside the implementation codebase, stage every change (including new files) and diff against the pinned commit. The `--cached` flag is what pulls newly created source files into the diff, even when the working tree is otherwise clean.

```
cd <workspace>/eclass-mcp-server
git add -A
git diff --cached dbd2d16 > ../solution.patch
```

2. Collect the transcripts. Claude Code stores transcripts under `~/.claude/projects/` in a per-workspace subdirectory whose name is the workspace's absolute path with every character that is not an ASCII letter, digit, or hyphen replaced by `-`. The path separator `/`, the dot `.`, and the underscore `_` all get replaced; usernames or directory names containing those characters are normalised the same way. List the entries and identify the directory belonging to your workspace by substring overlap with the workspace path:

```
ls ~/.claude/projects/
```

Inside that directory, the `.jsonl` files are your transcripts; the first record's `timestamp` field is the session's start, the last record's is its end. Pick the transcripts whose first timestamp falls inside the session window:

```
ls -la ~/.claude/projects/<encoded>/*.jsonl
head -1 ~/.claude/projects/<encoded>/<sessionId>.jsonl
tail -1 ~/.claude/projects/<encoded>/<sessionId>.jsonl
```

For each matching transcript, copy the `.jsonl` and the sibling per-session subdirectory of the same uuid (carrying the `subagents/` and `tool-results/` records) into `<workspace>/transcripts/`. Sessions that did not dispatch any subagents have no such subdirectory; copy only the `.jsonl` for those, and that is the correct shape:

```
mkdir -p <workspace>/transcripts
cp ~/.claude/projects/<encoded>/<sessionId>.jsonl \
    <workspace>/transcripts/
cp -r ~/.claude/projects/<encoded>/<sessionId>/ \
    <workspace>/transcripts/<sessionId>/
```

The third command runs only when `~/.claude/projects/<encoded>/<sessionId>/` exists; check with `ls -d ~/.claude/projects/<encoded>/<sessionId>/` first. Repeat for every transcript that belongs to the session, then verify:

```
ls -la <workspace>/transcripts
```

The directory should contain one `<sessionId>.jsonl` per main session and, optionally, one `<sessionId>/` subdirectory per session that dispatched subagents.

3. Package the submission. Zip `solution.patch` and `transcripts/` together at the workspace root. The filename uses your participant ID, the session number, and the framework you used in this session:

```
cd <workspace>
zip -r P001_S1_apm.zip solution.patch transcripts/
```

The zip contains exactly two entries at its root: `solution.patch` and `transcripts/`. Inside `transcripts/`: one `.jsonl` per main session at the top level, plus a subdirectory of the same uuid for each carrying the `subagents/` and `tool-results/` records.

4. Move the zip out of the workspace. The between-sessions reset wipes everything in the workspace, including the zip. Store it in a directory outside the workspace and outside the participant package; `~/Documents/bsc-apm-submissions/` is a reasonable default. Both session zips end up there, and the cleanup steps later leave that directory untouched.

```
mkdir -p ~/Documents/bsc-apm-submissions
mv <workspace>/P001_S1_apm.zip ~/Documents/bsc-apm-submissions/
```

5. Submit the form. Open the submission form, fill in the per-session questions, and attach the zip you just stored. Do this immediately after wrapping up, while the session is fresh.

https://forms.gle/yNDWvsBHAHwq8Dks7

8.4. Resetting between sessions

Run this after the first session has wrapped up and before the second session starts. Before running any of the steps below, confirm that session 1's submission zip is in your safe-storage directory (e.g., `ls -la ~/Documents/bsc-apm-submissions/` should list `P001_S1_*.zip`); the reset is destructive and only zips outside the workspace survive. The reset then returns the workspace and the transcript store to the same state the setup steps above produced for session 1, so session 2 begins from an identical baseline. The mock environment is left alone unless you have a reason to believe it has drifted from its initial state.

1. Reset the implementation codebase. From inside `eclass-mcp-server`, hard-reset to the pinned commit, wipe untracked and ignored files, and rebuild the virtual environment from scratch:

```
cd <workspace>/eclass-mcp-server
git reset --hard dbd2d16
git clean -fdx
uv sync --dev --all-extras
```

`git clean -fdx` removes every untracked file, including ignored files such as `.venv/` and any `__pycache__/` directories. Re-copy `.env` and `certs/` from the mock-environment prefix afterwards (Section 7.3); they are removed by `git clean`.

2. Reset the reference codebase. Do the same for `openeclass`, in case the AI inadvertently modified it:

```
cd <workspace>/openeclass
git reset --hard e8b3329
git clean -fdx
```

3. Sweep the workspace. Confirm the workspace root contains exactly four entries: `PRD.md`, `PROMPT.md`, `eclass-mcp-server/`, and `openeclass/`.

```
ls -la <workspace>
```

Delete any extra files or directories the AI may have created at the workspace root. If `PRD.md` or `PROMPT.md` were modified during the session, restore them from the participant package's `task/` directory; session 2 must read them in their original form.

4. Wipe the Claude Code project state. Remove this workspace's entry under `~/.claude/projects/` so session 2 starts from a clean baseline. The directory contains transcripts, per-session subdirectories, the project-local `memory/` store, and anything else Claude Code has cached for this workspace; the deletion is permanent.

```
ls -la ~/.claude/projects/<encoded>/
rm -rf ~/.claude/projects/<encoded>/
```

The `ls` step is the inventory; review it before running `rm`. Run only after you have safely stored the session's zip. Claude Code recreates the project directory on the next launch.

5. Reset the mock environment, only if needed. The mock environment retains its initial state across sessions and usually does not need a reset. Reset it only if a previous session modified the test data:

```
cd <mock>
./scripts/reset
```

6. Confirm the baseline. After the reset, confirm the workspace and the transcript store match the same state setup produced for session 1:

- `<mock>/scripts/status` reports every service ok.
- `<workspace>/eclass-mcp-server` is at `dbd2d16` with a clean working tree.
- `<workspace>/openeclass` is at `e8b3329` with a clean working tree.
- The workspace root contains `PRD.md`, `PROMPT.md`, `eclass-mcp-server/`, and `openeclass/`, and nothing else; `PRD.md` and `PROMPT.md` match their original contents from the participant package.
- Your workspace's project directory under `~/.claude/projects/` holds no session-1 state.

Once the baseline is in place, return to Section 8.1 and repeat the same starting-working-wrapping-up flow for session 2 (with your other assigned framework). Session 2 ends after Section 8.3, step 5 (*Submit the form*); the resetting steps are not needed to be run again.

8.5. Cleaning up

Optional. Once both sessions are wrapped up and submitted, nothing in this study needs to stay on your machine. If you want a clean slate:

- Wipe session 2's Claude Code project state: identify the workspace's entry under `~/.claude/projects/` as in Section 8.4 step 4 and `rm -rf` it.
- Delete the workspace: `rm -rf <workspace>`.
- Uninstall the mock environment: follow the steps in Section 7.4.

The participant package itself can be deleted at this point as well; it has no further role.

9. Contact

This study is conducted for a Bachelor's thesis by Konstantinos Chaidemenos (CobuterMan).

`sdi2200262@di.uoa.gr` Email

Thank you for your time and contribution to this research.